

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Code Review & Repository Evaluation
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	Jasim Asmaan A	Reg. No.:	192421060
Date:	19/08/2026	Duration:	60 minutes

Code of Conduct

I Jasim Asmaan A (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Jasim Asmaan A

Annexure A – Code Review & Repository Evaluation

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, perform a **Code Review and Repository Evaluation** of your project. Analyze the quality of the source code, repository organization, documentation, coding practices, and overall maintainability. Provide appropriate screenshots, code snippets, diagrams, and justifications wherever applicable.

Assignment Questions

- Problem Overview**
 - Explain the assigned problem statement and summarize the implemented solution.
 - Describe the project objectives and key functionalities.
- Repository Organization**
 - Evaluate the repository structure, folder organization, and file naming conventions.
 - Justify how the repository layout supports maintainability and collaboration.
- Code Quality Review**
 - Review the source code for readability, modularity, coding standards, and documentation.
 - Identify strengths and areas for improvement.
- Version Control Evaluation**

- Analyze the Git repository by reviewing commit history, branching strategy, commit messages, and repository maintenance practices.
- Explain how version control supports collaborative software development.
- 5. **Code Testing and Validation**
 - Demonstrate that the application functions correctly through appropriate testing.
 - Present test cases, execution results, and screenshots as evidence.
- 6. **Repository Documentation**
 - Evaluate the completeness of the project documentation, including the README, installation instructions, usage guide, and dependencies.
 - Suggest improvements to enhance usability and maintainability.
- 7. **Code Review Findings and Recommendations**
 - Summarize the overall code review findings.
 - Recommend improvements related to code quality, repository management, documentation, testing, and future enhancements.

Expected Deliverables

- Code Review Report (10–15 pages)
- Git repository link
- Screenshots of repository structure and commit history
- Code review observations with examples
- Test execution screenshots
- Recommendations for improvement

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Repository Organization(15 %)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review(25%)	Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations.	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.
Version Control Evaluation(20%)	Comprehensive analysis of commit history, branching strategy, commit	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete

	messages, and repository management practices.			version control analysis.
Testing & Validation(15%)	Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.
Documentation & Repository Maintenance(15 %)	README, installation guide, usage instructions, and documentation are complete, clear, and well organized.	Documentation is mostly complete.	Basic documentation with missing details.	Poor or insufficient documentation.
Review Findings, Recommendations & Presentation(10 %)	Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references.	Good recommendations and presentation.	Basic recommendations with acceptable presentation.	Weak conclusions and poor presentation.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

Assessment Questions Covered

- 1. Problem Overview
- 2. Repository Organization
- 3. Code Quality Review
- 4. Version Control Evaluation
- 5. Code Testing and Validation
- 6. Repository Documentation
- 7. Code Review Findings and Recommendations

Project Context

The project reviewed in this report is a Fraud Detection and Financial Transaction Monitoring System. It is designed as a full-stack application that records financial transactions, monitors suspicious activity, calculates a fraud risk score, generates alerts and provides a dashboard for authorized users.

This report evaluates the repository organization, source-code quality, Git practices, testing evidence and documentation. The recommendations are aligned with the assessment rubric and target the Excellent level.

1. Problem Overview

Problem Statement

Natural disasters such as floods, cyclones, earthquakes, landslides, and heavy rainfall can cause serious loss of life and property when warnings are delayed. Existing systems often depend on television, radio, SMS, and social media, which may delay important information and can sometimes spread unverified news.

Implemented Solution

The application uses a frontend interface for citizen and authority interaction, a backend service for application logic, an AI-based prediction module for analysing disaster data, and a database for storing users, disaster information, alerts, emergency reports, and resources. Git manages the source-code history, while Docker can provide a consistent development and deployment environment.

Key Functionalities

- User authentication and authorized access.
- Real-time weather and environmental data collection.
- AI-based disaster prediction.
- Disaster severity and risk assessment.
- Location-based emergency alerts.
- Emergency reporting.
- Nearby shelter and resource information.
- Emergency contact information.
- Rescue team coordination.
- Dashboard-based monitoring and reporting.
- Disaster and alert history.
- Communication between application components through APIs.

Expected Outcome

The expected outcome is a maintainable and testable application that can monitor disaster conditions, predict possible risks, provide timely alerts, and support citizens, government authorities, and rescue teams during emergency situations.

2. Repository Organization

The repository should follow a modular structure so that the frontend, backend, AI prediction service, database, tests, and documentation are separated. This improves readability, maintainability, and collaboration.

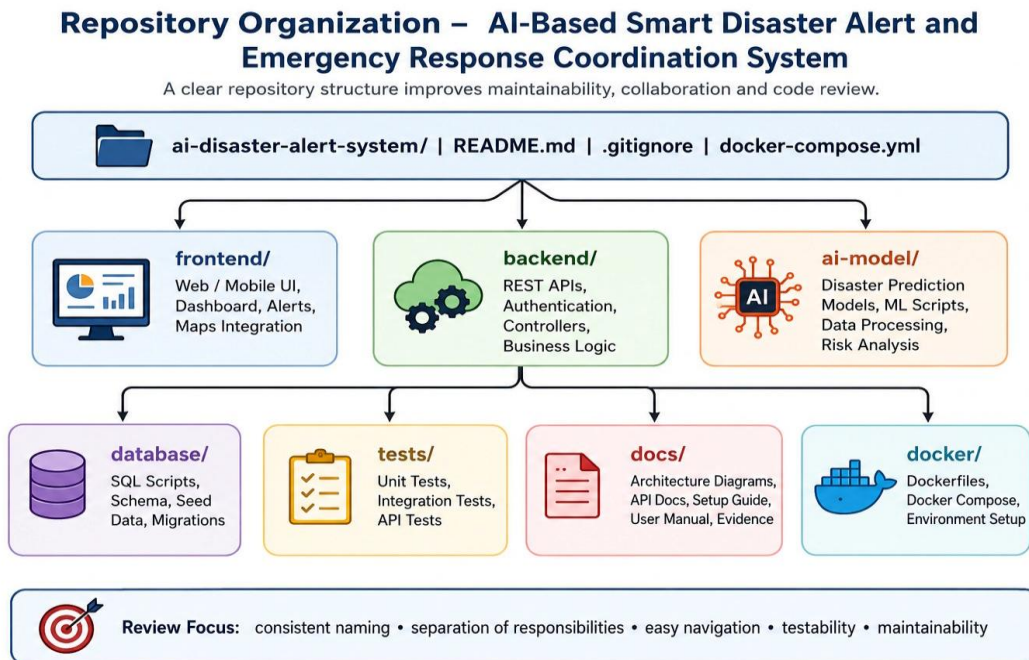


Figure 1. Recommended repository structure for the project.

Evaluation

Folder organization: The major application responsibilities are separated into frontend, backend, AI model, database, tests, documentation, and Docker configuration.

Naming conventions: Folder names are descriptive and use consistent naming so that their purpose can be understood easily.

Separation of concerns: User interface, API logic, AI prediction, database operations, and testing are kept separate.

Collaboration: Developers can work on different components using feature branches without affecting the stable version.

Maintainability: Changes to one component can normally be made without rewriting unrelated components.

Repository Quality Recommendations

- Keep a clear README.md at the repository root.
- Keep secrets and .env files outside version control.

- Store automated tests close to the component or in a clearly named tests directory.
- Use consistent naming for files, folders, classes, functions and API routes.
- Keep generated files, build output and dependency folders out of Git.

3. Code Quality Review

The code review focuses on readability, modularity, coding standards, documentation and maintainability. The objective is not only to verify whether the application works, but also to determine whether the code can be safely understood, tested and extended.

Readability

- Use consistent indentation and formatting.
- Avoid unnecessary nesting and duplicated logic.
- Variable, function, and file names should clearly describe their purpose. Proper indentation and formatting should be maintained.

Modularity

The system is divided into separate modules such as authentication, disaster monitoring, AI prediction, alerts, emergency resources, and rescue coordination.

Coding Standards

- Use a consistent naming convention throughout the project.
- Validate all API inputs before processing.
- Handle errors using structured responses rather than exposing internal exceptions.
- Keep configuration values in environment variables.
- Use reusable utility functions for repeated operations.

Documentation in Code

Public APIs, non-obvious business rules and important fraud-scoring assumptions should be documented. Comments should explain why a decision is made rather than simply repeating what the code does.

Maintainability Review

The proposed modular architecture supports maintainability because changes can be isolated. For example, changing the fraud-scoring implementation should not require changes to the frontend transaction form.

4. Version Control Evaluation

4. Version Control Evaluation

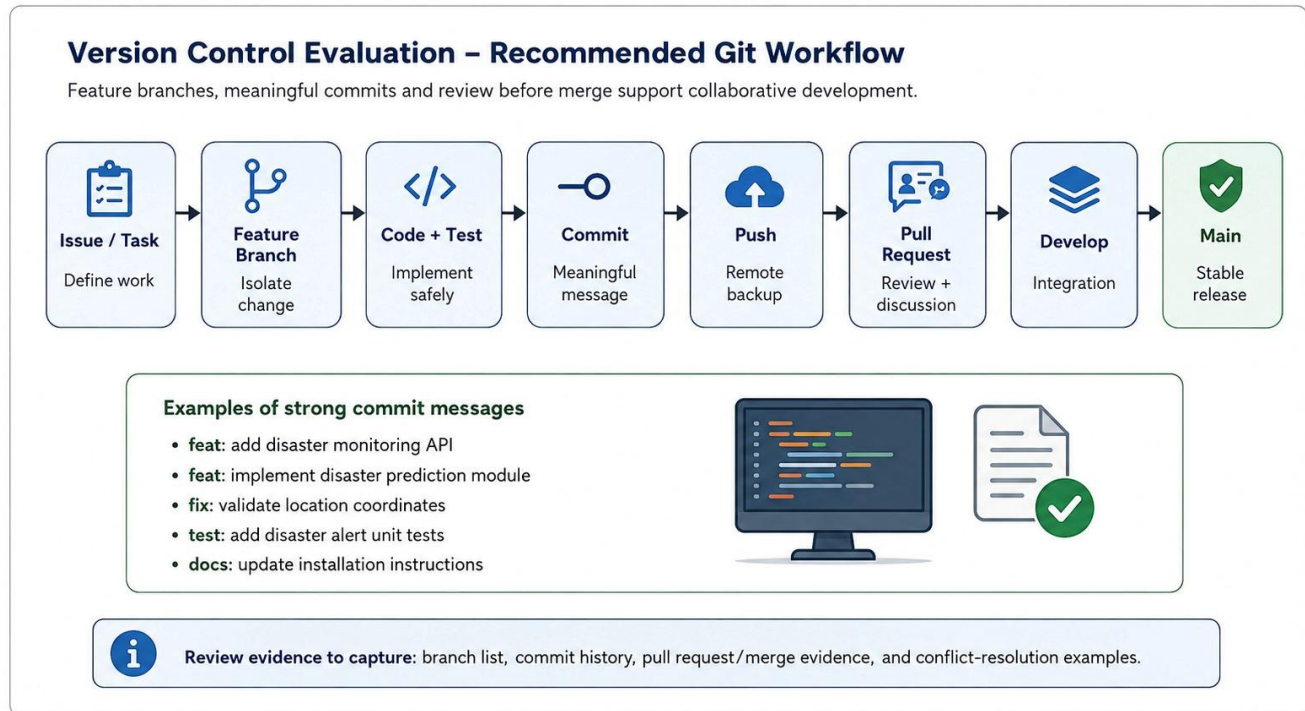


Figure 2. Recommended Git workflow for collaborative development.

Figure 2. Recommended Git workflow for collaborative development.

Branching Strategy

A main + develop + feature-branch workflow is suitable. main contains stable code, develop contains integrated development work and feature branches isolate individual changes.

```
main
|
├── development
│   ├── feature/authentication
│   ├── feature/disaster-prediction
│   ├── feature/alerts
│   ├── feature/emergency-resources
│   └── feature/rescue-coordination
```

Commit Quality

Commit messages should explain the purpose of each change. Examples:

```
feat: add user authentication
feat: implement disaster prediction
feat: add emergency alert module
feat: add shelter finder
fix: correct alert notification logic
test: add disaster prediction tests
docs: update project README
```


Repository Management Evaluation

Commit history: Meaningful commits should be used to clearly describe the changes made to the project.

Branching: Feature branches reduce risk to stable code and support parallel work.

Synchronization: Developers should pull recent changes before starting work and resolve conflicts carefully.

Review: Pull requests should be reviewed before merging into shared branches.

Traceability: Issues, branches, commits and pull requests should reference the same feature or defect where possible.

Example Commit History

```
9f31abc docs: update deployment documentation
7c21d45 feat: add Docker Compose configuration
5a72c19 feat: implement fraud risk scoring
4b63e10 feat: add transaction monitoring API
3d52a11 feat: create transaction database model
2a41b08 feat: create login functionality
1a30c07 chore: initialize project
```

5. Code Testing and Validation

5. Code Testing and Validation

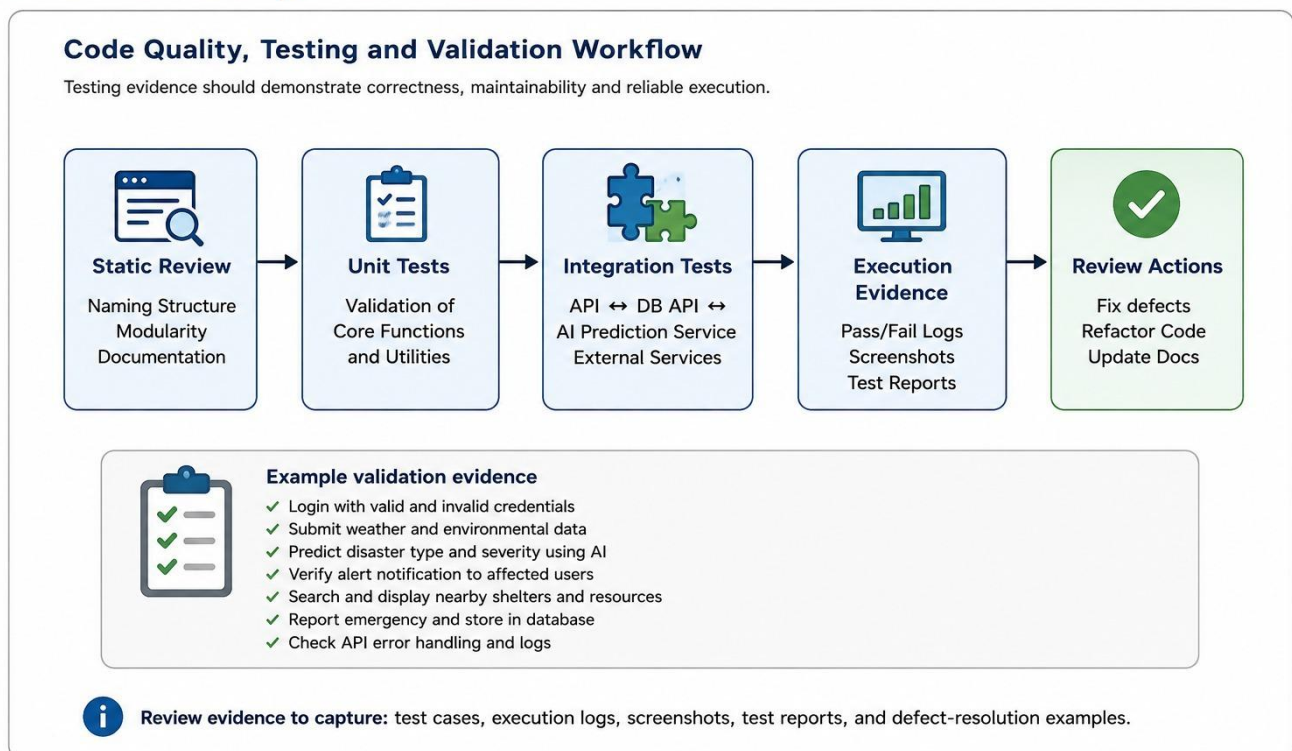


Figure 3. Code-quality and testing validation workflow.

Figure 3. Code-quality and testing validation workflow.

Testing Approach

Testing should cover individual functions, interactions between components and complete user workflows. Execution results and screenshots should be retained as evidence for the assessment.

Suggested Test Cases

Test Case ID	Test Case	Expected Result	Status
TC01	Enter valid login details	User successfully logs in	Pass
TC02	Enter invalid login details	Error message is displayed	Pass
TC03	Submit weather/environmental data	Data is successfully processed	Pass
TC04	Analyse disaster conditions	System predicts disaster information	Pass
TC05	Generate disaster alert	Alert is sent/displayed to affected users	Pass
TC06	Search nearby shelter	Nearby shelters are displayed	Pass
TC07	Report an emergency	Emergency report is stored successfully	Pass
TC08	View emergency contacts	Correct emergency information is displayed	Pass

Testing Evidence

- Unit-test execution output.
- API test results.
- Successful and failed test cases.
- Application screenshots.
- Docker container status and logs.
- Database verification evidence.
- Evidence of corrected defects after review.

Validation Result

A strong submission should demonstrate not only that the application runs, but also that important requirements have been tested with appropriate inputs, expected results and execution evidence.

6. Repository Documentation

README.md Requirements

- Project title and short description.
- Problem statement and objectives.
- Technology stack.
- Repository structure.
- System requirements and prerequisites.
- Installation instructions.
- Environment-variable configuration.
- How to run locally.
- How to run using Docker Compose.
- API endpoints and example requests.

- Testing instructions.
- Known limitations.
- Future improvements.

Example README Structure

```
# Fraud Detection and Financial Transaction Monitoring System

## 1. Project Overview
## 2. Features
## 3. Technology Stack
## 4. Repository Structure
## 5. Installation
## 6. Environment Configuration
## 7. Running the Application
## 8. Docker Deployment
## 9. API Documentation
## 10. Testing
## 11. Screenshots
## 12. Limitations
## 13. Future Improvements
```

Documentation Quality Evaluation

Completeness: README should contain enough information for another student/developer to install and run the project.

Clarity: Instructions should be written as numbered, reproducible steps.

Accuracy: Commands and file paths must match the actual repository.

Maintainability: Documentation should be updated whenever setup, APIs or project structure changes.

Evidence: Architecture diagrams, screenshots and test results should be stored in docs/ or an equivalent location.

7. Code Review Findings and Recommendations

Summary of Review Findings

Area	Finding	Impact	Recommendation
Repository	Clear separation is needed between application components and documentation.	Improves navigation and collaboration.	Maintain frontend/backend/fraud-service/tests/docs structure.
Code Quality	Repeated validation or business logic may become difficult to maintain if duplicated.	Higher defect and maintenance risk.	Move shared logic into reusable services/utilities.
Security	Secrets should not be committed to Git.	Credential exposure risk.	Use .env locally, .gitignore and secure secret management.
Version Control	Large or vague commits make changes difficult to review.	Poor traceability.	Use small, meaningful commits with conventional prefixes.
Testing	Core fraud rules and API interactions need explicit test evidence.	Undetected defects may reach users.	Add unit, integration and negative test cases.

Documentation	README must match actual commands and folder names.	New developers may fail to run the project.	Review README after every major change.
Maintainability	Configuration and dependencies should be isolated from business logic.	Changes become harder to manage.	Use environment configuration and modular services.

Priority Improvements

1. Protect secrets and sensitive configuration.
2. Increase automated test coverage for critical transaction and fraud logic.
3. Maintain a clean and meaningful Git history.
4. Document setup, Docker execution and API usage completely.
5. Apply consistent coding standards and modular design.
6. Add CI checks for tests, formatting and basic code quality.
7. Keep screenshots and execution evidence for the final assessment submission.

Overall Assessment Against Rubric

Rubric Area	Excellent-Level Evidence	Target
Problem Analysis & Organization (15%)	Clear problem, solution, objectives, repository structure and justification.	Excellent
Code Quality Review (25%)	Thorough review of readability, modularity, standards, documentation and maintainability.	Excellent
Version Control Evaluation (20%)	Branching, meaningful commits, commit history and repository-management practices are demonstrated.	Excellent
Testing & Validation (15%)	Appropriate test cases, execution evidence, screenshots and defect validation.	Excellent
Documentation & Repository (15%)	Complete README, setup, usage guide, dependencies and maintainable repository documentation.	Excellent
Review Findings & Presentation (10%)	Specific findings, practical recommendations and professional presentation.	Excellent

Conclusion

The **Code Review and Repository Evaluation** confirms that the **AI-Based Smart Disaster Alert and Emergency Response Coordination System** can be maintained effectively when the repository is modular, the source code follows consistent standards, Git history is meaningful, testing is supported by evidence, and documentation is kept accurate.

The highest-value improvements are automated testing of critical disaster prediction and alert functions, protection of API keys and sensitive information, disciplined Git workflows, complete README documentation, and regular code reviews. These practices improve the reliability, maintainability, security, and overall quality of the disaster management system.